Informe del Trabajo Práctico N°3

Juan I. Alaguibe, Juan F. Alvarez, Williams O. Condori Q., Marcos D. Lupotti

Facultad Regional Santa Fe UTN

Resumen: el presente informe resume las soluciones logradas por el grupo para responder a los incisos del segundo trabajo práctico de Análisis Numérico, así como las dificultades y retos que aparecieron.

Palabras claves: algoritmos, Newton-Raphson, bisección.

I. INTRODUCCIÓN

El presente informe recopila las dificultades y procedimientos que se llevaron a cabo para realizar la entrega del trabajo práctico N° 3 de Análisis Numérico. Se detalla brevemente cómo se implementaron las soluciones a los problemas planteados en el enunciado, y las diferentes iteraciones que se hicieron hasta llegar a la solución más satisfactoria. Además, se explicitan los problemas y retos que surgieron, y los descubrimientos a nivel implementación que los estudiantes encontraron.

II. DESARROLLO DE CONTENIDOS

1 Búsqueda de raíces en funciones no-lineales

1.a. Para este primer punto, utilizamos el método de Newton-Raphson combinado con el polinomio de Taylor de segundo orden (incluyendo la segunda derivada).

Para este algoritmo se requiere la función F(x) de la cual se desean hallar las raíces, junto con sus dos primeras derivadas, F′(x) y F″(x) (esto implica que la función debe ser dos veces derivable). Además, es necesario elegir un punto inicial, X□, en el que evaluar dichas funciones. Con estos elementos, se construye el polinomio de Taylor de grado 2 en el punto X□:

$$T(x) = F(X_n) + F'(X_n) * (x - X_n) + \frac{F''(X_n)}{2!} * (x - X_n)^2 \quad (1)$$

Este polinomio, siendo de grado 2, es una función cuadrática, lo que permite aprovechar las propiedades propias de estas funciones para el cálculo de raíces.

Si resolvemos $T(x) = 0$, expandiendo y reagrupando obtenemos:

$$a = \frac{1}{2}F''(x_n),\ b = F'(x_n) - x_n F''(x_n),\ c = F(x_n) - x_n F'(x_n) + \frac{1}{2}x_n^2 F''(x_n).$$

$$r_1 = \frac{-b+\sqrt{b^2-4ac}}{2a} = \frac{-F'(x_n)+x_nF''(x_n)+\sqrt{(F'(x_n)-x_nF''(x_n))^2-4*\frac{1}{2}F''(x_n)*(F(x_n)-x_nF'(x_n)+\frac{1}{2}x_n^2F''(x_n))}}{2\frac{1}{2}F''(x_n)} = \frac{-F'(x_n)+x_nF''(x_n)+\sqrt{[F'(x_n)]^2-2F(x_n)F''(x_n)}}{F''(x_n)}$$

(2)

$$r_2 = \frac{-b-\sqrt{b^2-4ac}}{2a} = \frac{-F'(x_n)+x_nF''(x_n)-\sqrt{(F'(x_n)-x_nF''(x_n))^2-4*\frac{1}{2}F''(x_n)*(F(x_n)-x_nF'(x_n)+\frac{1}{2}x_n^2F''(x_n))}}{2\frac{1}{2}F''(x_n)} = \frac{-F'(x_n)+x_nF''(x_n)-\sqrt{[F'(x_n)]^2-2F(x_n)F''(x_n)}}{F''(x_n)}$$

(3)

Asimismo, se aplica el método de Newton-Raphson, de acuerdo con la siguiente relación:

$$x_{n+1} = X_n - \frac{T(X_n)}{T'(X_n)} \quad (4)$$

Con esto, el algoritmo se puede describir de la siguiente manera:

Se parte de la función F(x), se calculan sus dos primeras derivadas y se selecciona un punto inicial X_n.

- Se computa el polinomio de Taylor (ecuación 1) en el punto X_n.
- Se determinan las raíces del polinomio T(x) utilizando las ecuaciones (2) y (3), y se calcula x_{n+1} mediante la ecuación (4).
- Si T(x) presenta raíces reales, se elige aquella que se encuentre más cercana a x_{n+1} y se asigna como el nuevo valor de X_n, sinó $X_n = X_{n+1}$.
- Cuando el discriminante es negativo o muy cercano a cero, las fórmulas (1) y (2) pueden dar un paso complejo o inestable. En ese caso, volvemos al paso clásico de Newton y nos quedamos con el valor calculado x_{n+1}:

El proceso se repite hasta que la diferencia $X_{n+1} - X_n$ sea menor al valor de tolerancia previamente establecido.

Condiciones de aplicación

Para poder aplicar este método se tiene que cumplir que la función sea dos veces derivable. Tanto $F(x)$, $F'(x)$ y $F''(x)$sean continuas y suaves.

Si en alguna iteración de X_n, $F''(X_n)$ El método falla ya que se hace una división por 0, y el polinomio de Taylor se asimila mucho a una recta con pendiente 0 por lo que tampoco se puede calcular el valor de la siguiente iteración calculando la raíz.

Determinar el error

Sea x^* la raíz exacta y $e_n = x_n - x^*$ el error en la iteración n

Al resolver exactamente el cuadrático de Taylor se anulan los términos hasta orden 2, y el primer término no nulo del residuo es de orden 3.

De ello se obtiene:

$$e_{n+1} = C\, e_n^3 + O(e_n^4), \quad C = -\frac{F'''(x^*)}{6\,F'(x^*)}$$

Por tanto, el término de error dominante es $O(e_n^3)$

Si en alguna iteración se hace fallback a Newton–Raphson clásico, en ese paso concreto el error cumple:

$$e_{n+1} = \frac{1}{2}\frac{F''(x^*)}{F'(x^*)}\, e_n^2 + O(e_n^3)$$

Por tanto, el término de error dominante es $O(e_n^{\ 2})$

Orden de convergencia

Como explicamos anteriormente, el orden de convergencia es p=3 (cúbico) siempre que se use la corrección cuadrática y cae a p=2 en los pasos donde se aplica el Newton clásico.

1.a.1 Generalizaciones

Luego de formalizar la solución encontrada, se decidió hacer una búsqueda más a fondo sobre los algoritmos de búsqueda de raíces. A continuación, se explicitan los métodos encontrados.

1.1.1 Método de Halley

El método de Halley es similar al método de Newton-Raphson que utiliza la derivada de segundo orden para lograr un orden de convergencia más alto. A diferencia de la aproximación cuadrática que empleamos en el inciso anterior donde se resuelve el polinomio completo y se elige una raíz explícita, Halley simplifica la fórmula al asumir una forma algebraicamente equivalente pero computacionalmente más estable, evitando raíces cuadradas.

Dado un punto x_n ,el método de Halley calcula la siguiente iteración como:

$$x_{n+1} = x_n - \frac{2f(x_n)f'(x_n)}{2(f'(x_n))^2 - f(x_n)f''(x_n)}$$

Esta fórmula surge de la resolución analítica del polinomio de Taylor de segundo orden, pero eligiendo una única rama de la raíz (la que se considera más estable). En este sentido, Halley automatiza la selección de raíz que, en nuestro caso, se hace manualmente al comparar cuál está más cerca de x_n.

Para aplicar correctamente el método de Halley se requiere:

- Que $f(x)$ sea dos veces derivable ($f \in C^2$).
- Que el denominador de la fórmula no se anule: $2(f'(x_n))^2 - f(x_n)f''(x_n) \neq 0$.

Además, debe tenerse cuidado cuando el denominador es muy pequeño, ya que puede inducir errores numéricos por pérdida de precisión, lo cual también motivó la estrategia de fallback a Newton-Raphson implementada en el algoritmo del inciso anterior. Por lo tanto, y aunque el método de Halley resulta ser una solución más estable y elegante, nuestra implementación brinda mayor control sobre la elección de raíces y permite intervenir cuando la estabilidad se ve comprometida.

1.1.2 Método de Householder

El método de Householder es una generalización de los métodos anteriores. La misma se expresa como:

$$x_{n+1} = x_n + p\frac{(1/f)^{(p-1)}(x_n)}{(1/f)^{(p)}(x_n)}$$

Donde, sí existe al p-ésima derivada de la función y es distinta de 0, es posible calcular su raíz de forma iterativa. Es observable que si $p = 1$ se obtiene el método de Newton-Raphson, y si $p = 2$ se obtiene el método de Halley. Si bien analíticamente es correcta y muy poderosa para tener métodos de mayor grado de convergencia, implica tener que derivar p veces a la función, por lo que se decidió no implementarla a nivel código por motivos de simplicidad.

Prueba de funcionamiento y comparación con Halley

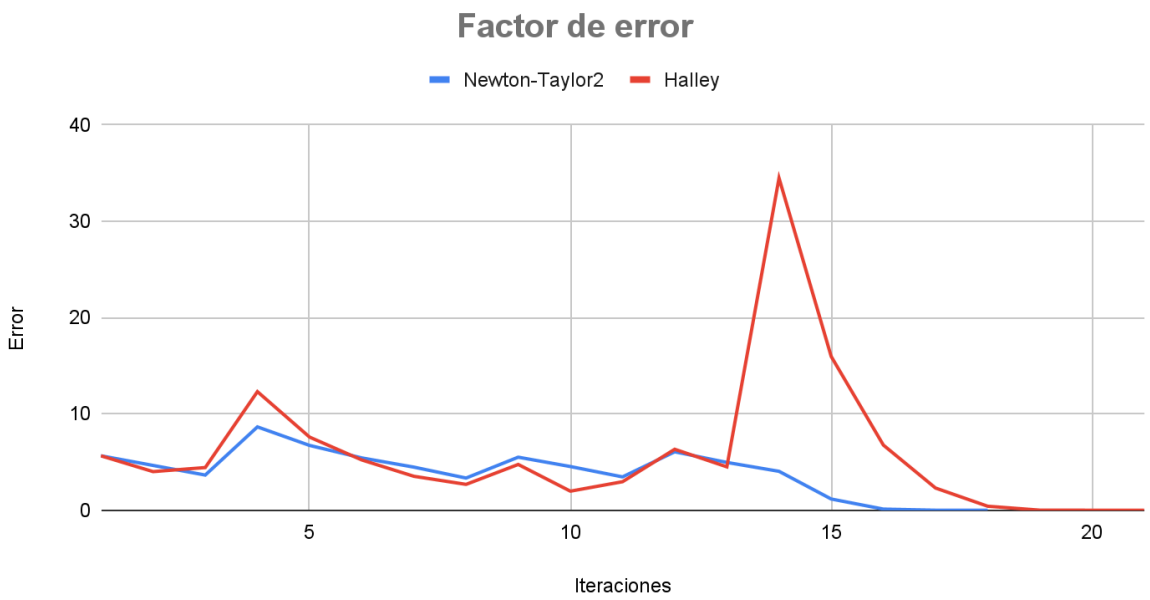
$F(x) = (x - 3)^3 - 4x + 20$

Newton-Taylor2:

```
Ingresa F(x) en sintaxis Python (e.g. 'x**3 - 2*x - 5'): (x-3)**3-4*x+20

Derivadas calculadas automáticamente:
 F'(x)  = 3*(x - 3)**2 - 4
 F''(x) = 6*x - 18

Valor inicial x0              (e.g. 2.0): 6
Tolerancia tol               (e.g. 1e-8): 1e-8
Máximo de iteraciones max_it (e.g. 30): 100
iter  1 → x=6.00000000 [NR ] → 5.00000000, Δ = -1.00000000,  F(x)=8.00e+00
iter  2 → x=5.00000000 [NR ] → 4.00000000, Δ = -1.00000000,  F(x)=5.00e+00
iter  3 → x=4.00000000 [NR ] → 9.00000000, Δ = 5.00000000,  F(x)=2.00e+02
iter  4 → x=9.00000000 [NR ] → 7.07692308, Δ = -1.92307692,  F(x)=5.95e+01
iter  5 → x=7.07692308 [NR ] → 5.78056429, Δ = -1.29635878,  F(x)=1.84e+01
iter  6 → x=5.78056429 [NR ] → 4.82322380, Δ = -0.95734050,  F(x)=6.77e+00
iter  7 → x=4.82322380 [NR ] → 3.69005699, Δ = -1.13316680,  F(x)=5.57e+00
iter  8 → x=3.69005699 [NR ] → 5.85550141, Δ = 2.16544441,  F(x)=1.99e+01
iter  9 → x=5.85550141 [NR ] → 4.88483583, Δ = -0.97066558,  F(x)=7.16e+00
iter 10 → x=4.88483583 [NR ] → 3.80989871, Δ = -1.07493711,  F(x)=5.29e+00
iter 11 → x=3.80989871 [NR ] → 6.41380929, Δ = 2.60391058,  F(x)=3.41e+01
iter 12 → x=6.41380929 [NR ] → 5.31151272, Δ = -1.10229657,  F(x)=1.11e+01
iter 13 → x=5.31151272 [NR ] → 4.38838363, Δ = -0.92312910,  F(x)=5.12e+00
iter 14 → x=4.38838363 [NR ] → 1.51501140, Δ = -2.87337222,  F(x)=1.07e+01
iter 15 → x=1.51501140 [cuad] → 0.23370521, Δ = -1.28130619,  F(x)=-2.10e+00
iter 16 → x=0.23370521 [cuad] → 0.35065786, Δ = 0.11695265,  F(x)=1.60e-03
iter 17 → x=0.35065786 [cuad] → 0.35056409, Δ = -0.00009378,  F(x)=-8.28e-13
iter 18 → x=0.35056409 [cuad] → 0.35056409, Δ = 0.00000000,  F(x)=7.11e-15

Convergió en 18 iteraciones. Raíz ≈ 0.35056409
```

Halley:

```
Ingresa F(x) en sintaxis Python (e.g. 'x**3 - 2*x - 5'): (x-3)**3-4*x+20

Derivadas calculadas automáticamente:
 F'(x)  = 3*(x - 3)**2 - 4
 F''(x) = 6*x - 18

Valor inicial x0              (e.g. 2.0): 6
Tolerancia tol               (e.g. 1e-8): 1e-8
Máximo de iteraciones max_it (e.g. 30): 100
iter  1 → x=6.00000000 [halley] → 4.35714286, Δ = -1.64285714,  F(x)=5.07e+00
iter  2 → x=4.35714286 [halley] → 4.77942769, Δ = 0.42228484,  F(x)=6.52e+00
iter  3 → x=4.77942769 [halley] → 12.65967372, Δ = 7.88024602,  F(x)=8.71e+02
iter  4 → x=12.65967372 [halley] → 7.94002346, Δ = -4.71965025,  F(x)=1.09e+02
iter  5 → x=7.94002346 [halley] → 5.57054962, Δ = -2.36947385,  F(x)=1.47e+01
iter  6 → x=5.57054962 [halley] → 3.87218498, Δ = -1.69836463,  F(x)=5.17e+00
iter  7 → x=3.87218498 [halley] → 3.03266424, Δ = -0.83952074,  F(x)=7.87e+00
iter  8 → x=3.03266424 [halley] → 5.10145209, Δ = 2.06878784,  F(x)=8.87e+00
iter  9 → x=5.10145209 [halley] → 2.32717083, Δ = -2.77428126,  F(x)=1.04e+01
iter 10 → x=2.32717083 [halley] → 3.30912095, Δ = 0.98195012,  F(x)=6.79e+00
iter 11 → x=3.30912095 [halley] → 6.67728127, Δ = 3.36816031,  F(x)=4.30e+01
iter 12 → x=6.67728127 [halley] → 4.85374834, Δ = -1.82353292,  F(x)=6.96e+00
iter 13 → x=4.85374834 [halley] → -34.12384032, Δ = -38.97758866,  F(x)=-5.10e+04
iter 14 → x=-34.12384032 [halley] → -15.61120724, Δ = 18.51263308,  F(x)=-6.36e+03
iter 15 → x=-15.61120724 [halley] → -6.41277622, Δ = 9.19843103,  F(x)=-7.88e+02
iter 16 → x=-6.41277622 [halley] → -1.95319774, Δ = 4.45957847,  F(x)=-9.37e+01
iter 17 → x=-1.95319774 [halley] → -0.06374230, Δ = 1.88945545,  F(x)=-8.50e+00
iter 18 → x=-0.06374230 [halley] → 0.34261439, Δ = 0.40635669,  F(x)=-1.36e-01
iter 19 → x=0.34261439 [halley] → 0.35056401, Δ = 0.00794961,  F(x)=-1.35e-06
iter 20 → x=0.35056401 [halley] → 0.35056409, Δ = 0.00000008,  F(x)=-7.11e-15
iter 21 → x=0.35056409 [halley] → 0.35056409, Δ = 0.00000000,  F(x)=7.11e-15

Convergió en 21 iteraciones. Raíz ≈ 0.35056409
```



1.b. Para este punto utilizamos la implementación ya explicada en el punto 1.a, añadiendo el método de la bisección. Este método se basa en tomar dos puntos, "a" y "b", dentro del dominio de la función y se evalúan en la función, si ambos son de distinto signo, es porque entre ellos se encuentra una raíz de la función. Se hace la operación $\frac{(a+b)}{2}$ y el resultado se toma como X_{n+1}. A X_{n+1} se le compara el signo con a y b y reemplaza el valor del que posea el mismo signo, quedando un intervalo, por ejemplo, $[a: X_{n+1}]$. Estos pasos se repiten hasta que la diferencia entre un extremo del intervalo y el otro sea menor a la tolerancia deseada o que el valor de f(x_{n+1}) sea lo suficientemente cercana a 0.

La manera en la cual implementamos los dos algoritmos es la siguiente:

- Del intervalo dado se toma el punto medio x_n
- Se comprueba que la primera derivada $f'(x_{n+1}) \neq 0$
- Se calcula el polinomio de Taylor $T(x)$ con sus raíces(2) y (3) y se calcula x_{n+1} mediante la ecuación (4).
- Si $T(x)$ presenta raíces reales, se elige aquella que se encuentre más cercana a x_{n+1} y se asigna como el nuevo valor de X_n, sinó $X_n = X_{n+1}$.
- Este X_{n+1} se evalúa y se intercambia por uno de los extremos del intervalo que tenga el mismo signo.
- Si $f'(x_{n+1}) \approx 0$ se pasa al método de la bisección, en la cual simplemente se compara con a y b y se asigna X_n a uno de los extremos del intervalo que tenga el mismo signo.

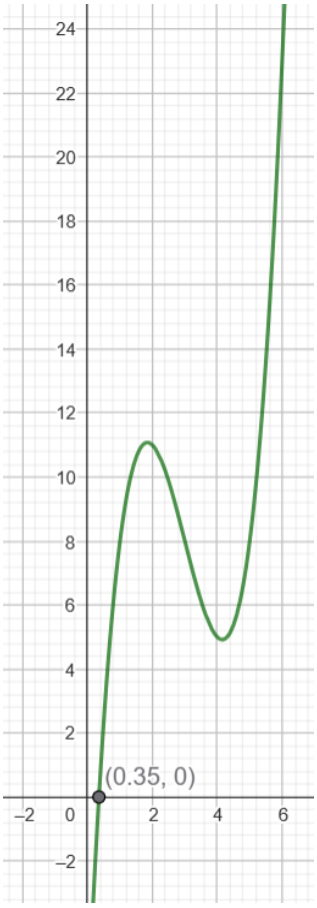
1.b.1 Observaciones

Una limitación que tenemos en los algoritmos anteriores es cuando en una iteración $f'(x_n) = 0$, ya que a la hora de calcular x_{n+1} con Newton o Halley nos encontramos con una división por cero y haciendo que el cálculo de raíces también sea indefinido, lo que nos quita la posibilidad de seguir con el algoritmo tal como está diseñado. Esto lo pudimos solucionar implementando bisección, haciendo que cuando sucede este escenario, simplemente seleccionamos el valor intermedio del intervalo como siguiente x_{n+1}.

El coste computacional de usar bisección es casi nulo comparado al cálculo de derivadas, ya que simplemente es una resta y división seguidos por una comparación. Sumado a la convergencia que logramos con Newton-Cuadratico, se logra una implementación de muy pocas iteraciones y bajo cómputo.

Prueba de funcionamiento y comparación con Newton-Taylor2

$F(x) = (x - 3)^3 - 4x + 20$



Newton-Taylor2:

```
Ingresa F(x) en sintaxis Python (e.g. 'x**3 - 2*x - 5'): (x-3)**3-4*x+20

Derivadas calculadas automáticamente:
 F'(x)  = 3*(x - 3)**2 - 4
 F''(x) = 6*x - 18

Valor inicial x0              (e.g. 2.0): 6
Tolerancia tol               (e.g. 1e-8): 1e-8
Máximo de iteraciones max_it (e.g. 30): 100
iter  1 → x=6.00000000 [NR ] → 5.00000000, Δ = -1.00000000,  F(x)=8.00e+00
iter  2 → x=5.00000000 [NR ] → 4.00000000, Δ = -1.00000000,  F(x)=5.00e+00
iter  3 → x=4.00000000 [NR ] → 9.00000000, Δ = 5.00000000,  F(x)=2.00e+02
iter  4 → x=9.00000000 [NR ] → 7.07692308, Δ = -1.92307692,  F(x)=5.95e+01
iter  5 → x=7.07692308 [NR ] → 5.78056429, Δ = -1.29635878,  F(x)=1.84e+01
iter  6 → x=5.78056429 [NR ] → 4.82322380, Δ = -0.95734050,  F(x)=6.77e+00
iter  7 → x=4.82322380 [NR ] → 3.69005699, Δ = -1.13316680,  F(x)=5.57e+00
iter  8 → x=3.69005699 [NR ] → 5.85550141, Δ = 2.16544441,  F(x)=1.99e+01
iter  9 → x=5.85550141 [NR ] → 4.88483583, Δ = -0.97066558,  F(x)=7.16e+00
iter 10 → x=4.88483583 [NR ] → 3.80989871, Δ = -1.07493711,  F(x)=5.29e+00
iter 11 → x=3.80989871 [NR ] → 6.41380929, Δ = 2.60391058,  F(x)=3.41e+01
iter 12 → x=6.41380929 [NR ] → 5.31151272, Δ = -1.10229657,  F(x)=1.11e+01
iter 13 → x=5.31151272 [NR ] → 4.38838363, Δ = -0.92312910,  F(x)=5.12e+00
iter 14 → x=4.38838363 [NR ] → 1.51501140, Δ = -2.87337222,  F(x)=1.07e+01
iter 15 → x=1.51501140 [cuad] → 0.23370521, Δ = -1.28130619,  F(x)=-2.10e+00
iter 16 → x=0.23370521 [cuad] → 0.35065786, Δ = 0.11695265,  F(x)=1.60e-03
iter 17 → x=0.35065786 [cuad] → 0.35056409, Δ = -0.00009378,  F(x)=-8.28e-13
iter 18 → x=0.35056409 [cuad] → 0.35056409, Δ = 0.00000000,  F(x)=7.11e-15

Convergió en 18 iteraciones. Raíz ≈ 0.35056409
```

Newton-Taylor2-Bisección:

```
Ingresa F(x) en sintaxis Python: (x-3)**3-4*x+20

Derivadas: F'(x)=3*(x - 3)**2 - 4, F''(x)=6*x - 18

Extremo a (F(a)*F(b)<0): 0
Extremo b: 12
Tolerancia en x (e.g. 1e-8): 1e-8
Tolerancia en f (e.g. 1e-8): 1e-8
Máximo iteraciones: 100
iter  1 → x=6.00000000 [NR ] → 5.00000000, Δ = -1.00000000,  F(x)=8.00e+00
iter  2 → x=5.00000000 [NR ] → 4.00000000, Δ = -1.00000000,  F(x)=5.00e+00
iter  3 → x=4.00000000 [bisec] → 2.00000000, Δ = -2.00000000,  F(x)=1.10e+01
iter  4 → x=2.00000000 [bisec] → 1.00000000, Δ = -1.00000000,  F(x)=8.00e+00
iter  5 → x=1.00000000 [cuad] → 0.33333333, Δ = -0.66666667,  F(x)=-2.96e-01
iter  6 → x=0.33333333 [cuad] → 0.35056439, Δ = 0.01723105,  F(x)=5.12e-06
iter  7 → x=0.35056439 [cuad] → 0.35056409, Δ = -0.00000030,  F(x)=7.11e-15

Convergió en 7 iteraciones por |F(x)|<tol_f. Raíz≈0.35056409
```

2.Aplicación de sistema de gases

2.a. Correcciones introducidas por el modelo de Van der Waals

La ley de gases ideales se basa en una serie de supuestos simplificadores: que las moléculas no interaccionan entre sí y que no ocupan espacio. Esto lleva a una buena aproximación para gases a baja presión y alta temperatura. Sin embargo, cuando se trabaja con gases reales**,** estos supuestos dejan de ser válidos. En 1873, Johannes Diderik Van der Waals propuso una corrección a la ecuación de estado de los gases que considera los dos factores más importantes que afectan el comportamiento de un gas real.

- La fuerzas intermoleculares (atractivas)
- El volumen propio de las moléculas.

La ecuación propuesta es:

$$(P + \frac{a}{v^2})(v - b) = RT$$

Donde:

- v: volúmen molar (m^3/mol),
- a: coeficiente que corrige las interacciones atractivas entre moléculas ($Pa.m^6/mol^2$)
- b: volumen propio de las moléculas por mol (m^3)
- R: constante universal de los gases, $R = 8,314\, J/mol.K$
- T: temperatura en Kelvin
- P: presión en pascales

2.a.1 Correcciones de la presión : $\frac{a}{v^2}$

Cuando un gas está a alta densidad (bajo volumen molar), las moléculas están muy próximas entre sí. Estos producen fuerzas atractivas entre ellas, que hacen que el impacto efectivo sobre las paredes del recipiente sea menor al que predecía la ley ideal. Es decir, la presión real es menor que la calculada por el modelo ideal.

Van der Waals propuso entonces sumar un término $\frac{a}{v^2}$**,** con $a = 0,364\, Pa * m^6/mol^2$, a la presión para compensar esta diferencia. Cuanto menor es el volumen molar, más importante es este término.

2.a.2 Corrección del volumen: b

El modelo ideal supone que las moléculas son puntos sin volumen. Sin embargo, cada molécula ocupa espacio físico. Esto significa que el volumen disponible para el movimiento de las moléculas es menor que el volumen total del recipiente. Van der Waals corrigió esto reemplazando el volumen $v - b$**,** donde $b = 4,27 * 10^{-5}\, m^3/mol$,que representa el volumen excluido por mol de gas.

2.a.3 Cálculo del volumen molar real según Van der Waals.

Se plantea la función:

$$f(v) = (P + \frac{a}{v^2})(v - b) - RT = 0$$

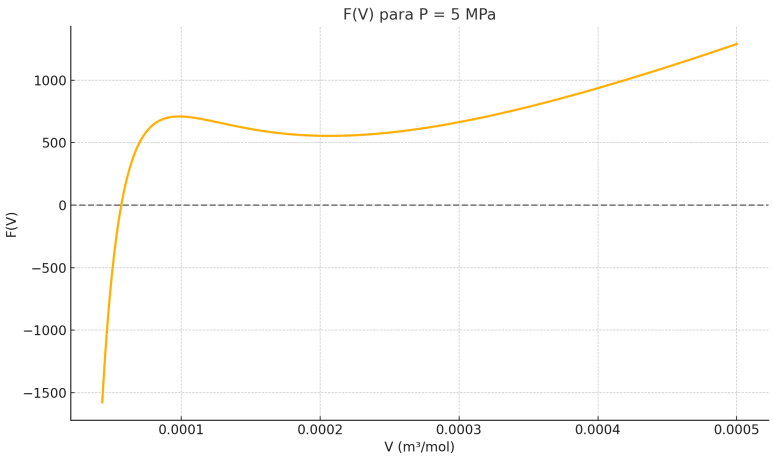
Con la que calcularemos

Enunciado a. "Determinar el volumen real que tendría el gas bajo esas condiciones".

$$f(v) = (P + \frac{a}{v^2})(v - b) - RT = 0$$

1-Primera derivada $f'(v)$: $f'(v) = P - \frac{2a(v-b)}{v^3} + (P + \frac{a}{v^2})$

2- Segunda derivada $f''(v)$: $f''(v) = \frac{6a(v-b)}{v^4} - \frac{4a}{v^3}$




```
a = 0.364              # Pa·m^6/mol^2
b = 4.27e-5            # m^3/mol
R = 8.314462618        # J/(mol·K)
T = 200                # K
P = 5e6                # P
```

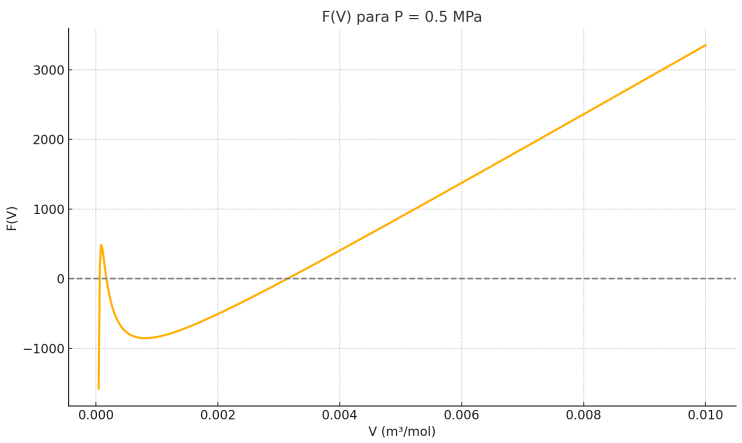
Se aplicó un método combinado Newton-Raphson + Taylor, alternando entre ambos según la derivación y la calidad del paso, utilizamos como condición inicial el valor del volumen calculado con la fórmula de la ley de los gases ideales:

$$x_0 = \frac{R*T}{P} = \frac{8.314x\,200}{5x10^6} = 0.00033\ m^3/mol$$

```
iter  1 → x = 0.00033258, [NR ] Δ = -0.00028990, xₙ₊₁ = 0.00004267
iter  2 → x = 0.00004267, [NR ] Δ = 0.00000813, xₙ₊₁ = 0.00005081
iter  3 → x = 0.00005081, [cuad] Δ = 0.00000652, xₙ₊₁ = 0.00005733
iter  4 → x = 0.00005733, [cuad] Δ = -0.00000050, xₙ₊₁ = 0.00005683
iter  5 → x = 0.00005683, [cuad] Δ = 0.00000000, xₙ₊₁ = 0.00005683

Convergió en 5 iteraciones. Raíz ≈ 0.00005683
```

Luego de aplicar el algoritmo iterativo, se identificó una única raíz real de la función $f(v)$, lo que indica que el gas se encuentra en una fase condensada. El valor hallado del volumen molar fue de $5,683x10^{-5}m^3/mol$, notablemente inferior al obtenido mediante la ley de los gases ideales, confirmando la relevancia de las correcciones introducidas por Van der Waals.

Enunciado b. "Determinar nuevamente el volumen real del gas utilizando ahora una presión de 0.5 MPa utilizando los dos métodos desarrollados en el insiso anterior. Evalúe diferentes condiciones iniciales. Obtener conclusiones a partir de los resultados".




```
a = 0.364            # Pa·m^6/mol^2
b = 4.27e-5          # m^3/mol
R = 8.314462618      # J/(mol·K)
T = 200              # K
P = 0.5e6            # P
```

Para $T = 200K$ y $P = 5\times10^5$ hay 3 raíces reales:

$V_1 \approx 5.80\times10^{-5}(m^3/mol)$, $V_2 \approx 1.71\times10^{-4}(m^3/mol)$, $V_3 \approx 3.14\times10^{-3}(m^3/mol)$

Interpretación física

La raíz más grande $V_3 \approx 3.14x10^{-3}(m^3/mol)$ corresponde a la fase gaseosa estable, ya que representa el volumen característico de moléculas en estado disperso y se encuentra en una región donde la pendiente $\vartheta P/\vartheta V < 0$,condición necesaria para la estabilidad mecánica. Las otras raíces, de menor volumen, se asocian a una fase condensada (líquida) o a una zona inestable (como la raíz intermedia V_2que) no representa un estado observable físicamente.

Condición inicial 1

Utilizamos como condición inicial el valor del volumen calculado con la fórmula de la ley de los gases ideales:

$$x_0 = \frac{R*T}{P} = \frac{8.314x\,200}{5x10^6} = 0.0033257850472\ m^3/mol$$

Utilizando el primer método desarrollado: (Newton-Taylor2)

```
Condición inicial: 0.0033257850472
iter  1 → x = 0.00332579, [cuad] Δ = -0.00018597, xₙ₊₁ = 0.00313982
iter  2 → x = 0.00313982, [cuad] Δ = -0.00000004, xₙ₊₁ = 0.00313977
iter  3 → x = 0.00313977, [cuad] Δ = -0.00000000, xₙ₊₁ = 0.00313977

Convergió en 3 iteraciones. Raíz ≈ 0.00313977
```

Utilizando el segundo método desarrollado: (Newton-Taylor2 + bisección)

Utilizamos un intervalo centrado en el valor obtenido de la fórmula de gases ideales $0.0033257850472 \pm 0.001$, redondeando nos queda a=0.002 y b=0.004

```
Ingresa F(x) en sintaxis Python: (5e5+0.364/x**2)*(x-4.27e-5)-8.314462618*200

Derivadas: F'(x)=500000.0 + 0.364/x**2 - 0.728*(x - 4.27e-5)/x**3, F''(x)=-1.456/x**3 + 2.184*(x - 4.27e-5)/x**4

Extremo a (F(a)*F(b)<0): 0.002
Extremo b: 0.004
Tolerancia en x (e.g. 1e-8): 1e-8
Tolerancia en f (e.g. 1e-8): 1e-8
Máximo iteraciones: 100
iter  1 → x=0.00300000 [cuad] → 0.00313975, Δ = 0.00013975,  F(x)=-1.11e-02
iter  2 → x=0.00313975 [cuad] → 0.00313977, Δ = 0.00000002,  F(x)=-4.55e-13

Convergió en 2 iteraciones por |F(x)|<tol_f. Raíz≈0.00313977
```

Vemos que en este caso converge a la raíz $V_3 \approx 3.14 \times 10^{-3} (m^3/mol)$ la cual físicamente consideramos correcta. Y de la cual utilizando este método obtenemos un mayor acercamiento a su valor con varios decimales.

Condición inicial 2

Utilizamos como condición inicial el valor 1: x0=6e-5

```
Condición inicial: 6e-05
iter  1 → x = 0.00006000, [cuad] Δ = -0.00000201, xₙ₊₁ = 0.00005799
iter  2 → x = 0.00005799, [cuad] Δ = 0.00000001, xₙ₊₁ = 0.00005799

Convergió en 2 iteraciones. Raíz ≈ 0.00005799
```

Vemos que en este caso converge a la raíz $V_1 \approx 5.80 \times 10^{-5} (m^3/mol)$ nombrada anteriormente.

Condición inicial 3

x0=9e-5

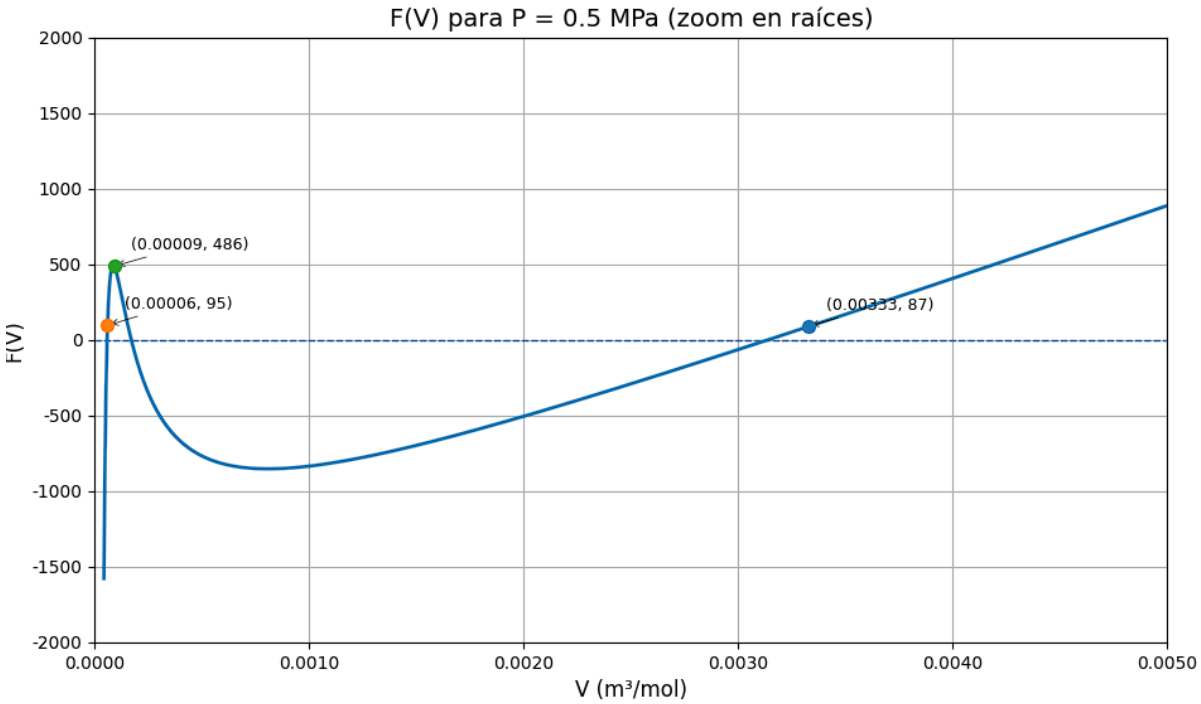
```
Condición inicial: 9e-05
iter  1 → x = 0.00009000, [cuad] Δ = 0.00004390, xₙ₊₁ = 0.00013390
iter  2 → x = 0.00013390, [cuad] Δ = 0.00003538, xₙ₊₁ = 0.00016928
iter  3 → x = 0.00016928, [cuad] Δ = 0.00000144, xₙ₊₁ = 0.00017072
iter  4 → x = 0.00017072, [cuad] Δ = 0.00000000, xₙ₊₁ = 0.00017072

Convergió en 4 iteraciones. Raíz ≈ 0.00017072
```

Vemos que en este caso converge a la raíz $V_2 \approx 1.71 \times 10^{-4} (m^3/mol)$ nombrada anteriormente.

Análisis gráfico

Para el punto azul utilizamos como condición inicial el valor del volumen calculado con la fórmula de la ley de los gases ideales:

$$x_0 = V_0 = \frac{R*T}{P} = \frac{8.314x\,200}{5x10^6} = 0.0033257850472\ m^3/mol$$



Donde:

- El punto verde representa el x_0 del caso 3, que converge a la raíz V_2.
- El punto naranja representa el x_0 del caso 2, que converge a la raíz V_1.
- El punto azul representa el x_0 del caso 1, que converge a la raíz V_3.

Esto nos deja claro que lo más importante a la hora de aplicar estos métodos es elegir un buen punto de partida o intervalo donde empezar a buscar, porque eso nos lleva a encontrar el valor correcto, en este caso el valor con sentido físico correcto.

Para la condición inicial 1, realizamos la comparación de ambos códigos (newton-taylor2 y newton-taylor 2+bisección) y verificamos que ambos nos conducen a la misma solución, aunque el método que incluye bisección es más eficiente debido al intervalo acotado de búsqueda que además nos proporciona una mayor seguridad en la obtención de el valor correcto de la raíz buscada.

III. CONCLUSIONES

Como conclusiones finales, se pudo llegar a una correcta resolución de los enunciados del trabajo práctico. Puntualmente, se destaca que:

- Es posible, a partir de las raíces del polinomio resultante de la expansión de Taylor, conseguir un algoritmo de orden de convergencia cúbico para encontrar las raíces de una función. Se aprovechan las raíces de la expansión, y se mejora la convergencia del algoritmo con Newton-Raphson.
- El trabajo permitió reforzar la comprensión de los métodos de búsqueda de raíces no lineales, su comportamiento numérico, y su importancia en la modelización de fenómenos físicos reales.
- La combinación de los métodos de alta convergencia y los métodos de intervalos seguros generan soluciones muy eficientes y confiables.
- El análisis gráfico evidenció la sensibilidad del método iterativo respecto al valor inicial. Este punto resulta fundamental al aplicar métodos numéricos a ecuaciones con múltiples soluciones, ya que afecta directamente la interpretación física del resultado.

IV. BIBLIOGRAFÍA

[1] “Newton’s method and high order iterations,” Free.fr, 2025. http://numbers.computation.free.fr/Constants/Algorithms/newton.html (accessed Jun. 25, 2025).

[2] "Método de Newton," *Wikipedia: La enciclopedia libre*. [En línea]. Disponible en: https://es.wikipedia.org/wiki/M%C3%A9todo_de_Newton (accessed Jun. 25, 2025)

[3] "Bisection method," *Wikipedia*. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Bisection_method (accessed Jun. 25, 2025)

[4] "Halley's method," *Wikipedia*. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Halley%27s_method (accessed Jun. 25, 2025)

[5] "Householder's method," *Wikipedia*. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Householder%27s_method (accessed Jun. 25, 2025)

[6] "Taylor series," *Wikipedia*. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Taylor_series (accessed Jun. 25, 2025)